

AI ENGINEER ROADMAP

Curso Personalizado de AI Engineering

André Rizzato

Projeto base: DistributedOrderSystem

Volume 1 - Semana 1

Setup de Ambiente

Antes de escrever qualquer linha de IA, preparamos o terreno. A camada de IA do DistributedOrderSystem vive dentro de uma pasta **ai/** separada da solução .NET, para não misturar responsabilidades: o C# continua sendo a camada de domínio, o Python entra como camada de inteligência.

Passo 1 - Ambiente Python

Confirme a versão instalada e instale as duas dependências iniciais do curso: o SDK oficial da Anthropic e o utilitário para carregar variáveis de ambiente.

```
python --version
pip install anthropic python-dotenv
```

Comandos de terminal - Semana 1

Passo 2 - Estrutura de pastas

```
DistributedOrderSystem/
  ai/
    .env                <- ANTHROPIC_API_KEY (no .gitignore)
    .gitignore
  week1/
    first_call.py       <- exercicio atual
    streaming_call.py   <- proximo passo
```

Estrutura de arquivos da Semana 1

Passo 3 - Chave de API e segurança

A chave é obtida em **console.anthropic.com** → **API Keys** → **Create Key**. Ela nunca vai para o Git: o arquivo **.env** guarda a chave localmente e o **.gitignore** impede que ela seja versionada.

```
# .env
ANTHROPIC_API_KEY=sua_chave_aqui

# .gitignore
.env
__pycache__/
*.pyc
```

Conteúdo de .env e .gitignore

Fundamentos de LLM

Um **LLM** (Large Language Model) é um modelo estatístico treinado para prever a próxima unidade de texto (token) dado o contexto anterior. Isso é suficiente, em escala, para gerar texto coerente, responder perguntas, escrever código e raciocinar sobre problemas.

Tokens e janela de contexto

Um **token** é a unidade mínima de texto que o modelo processa - em média, algo entre 3 e 4 caracteres em português. A **context window** é o número máximo de tokens (entrada + saída) que o modelo consegue considerar em uma única chamada. Todo o histórico de conversa, o system prompt e a pergunta atual competem pelo mesmo espaço.

System prompt vs. user prompt

O **system prompt** define o papel, as regras e o tom do assistente antes da conversa começar - é onde configuramos, por exemplo, que o assistente deve responder sempre em português e conhecer o contexto do DistributedOrderSystem. O **user prompt** é a mensagem específica do usuário em cada turno.

Temperature

Controla a aleatoriedade da geração: valores baixos (perto de 0) tornam a resposta mais determinística e previsível - ideal para tarefas de negócio como classificar um pedido. Valores altos aumentam a criatividade/diversidade, mas também o risco de respostas menos precisas.

Prévia: embeddings

Além de gerar texto, um modelo pode transformar texto em um vetor numérico (**embedding**) que representa seu significado. Dois textos com significados próximos geram vetores próximos no espaço. Essa ideia é a base do RAG, que construiremos do zero ainda na Fase 1.

Anthropic API na Prática

Toda interação com o modelo passa pelo endpoint **messages.create**. Os parâmetros centrais são:

- **model** - qual modelo será usado (ex: claude-sonnet-4-6)
- **max_tokens** - limite de tokens de saída
- **system** - o system prompt (papel do assistente)
- **messages** - lista de turnos da conversa (role + content)

Exercício: first_call.py

Este é o primeiro script que André executa no curso - uma chamada simples, com system prompt já ancorado no contexto do DistributedOrderSystem.

```
# week1/first_call.py
import os
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

client = Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system="""Voce e um assistente especializado no sistema DistributedOrderSystem.
    Esse sistema gerencia pedidos distribuidos com microservicos em .NET/C#.
    Responda sempre em portugues.""",
    messages=[
        {"role": "user", "content": "O que e um sistema de pedidos distribuido e
        quais sao os principais desafios?"}
    ]
)

print(response.content[0].text)
print(f"\n--- Uso de tokens ---")
print(f"Input:  {response.usage.input_tokens}")
print(f"Output: {response.usage.output_tokens}")
```

Código completo - week1/first_call.py

O que observar na resposta

- O modelo respondeu dentro do contexto do system prompt?
- Quantos tokens de input foram usados - o system prompt já custa tokens
- A latência - tempo até a primeira palavra aparecer

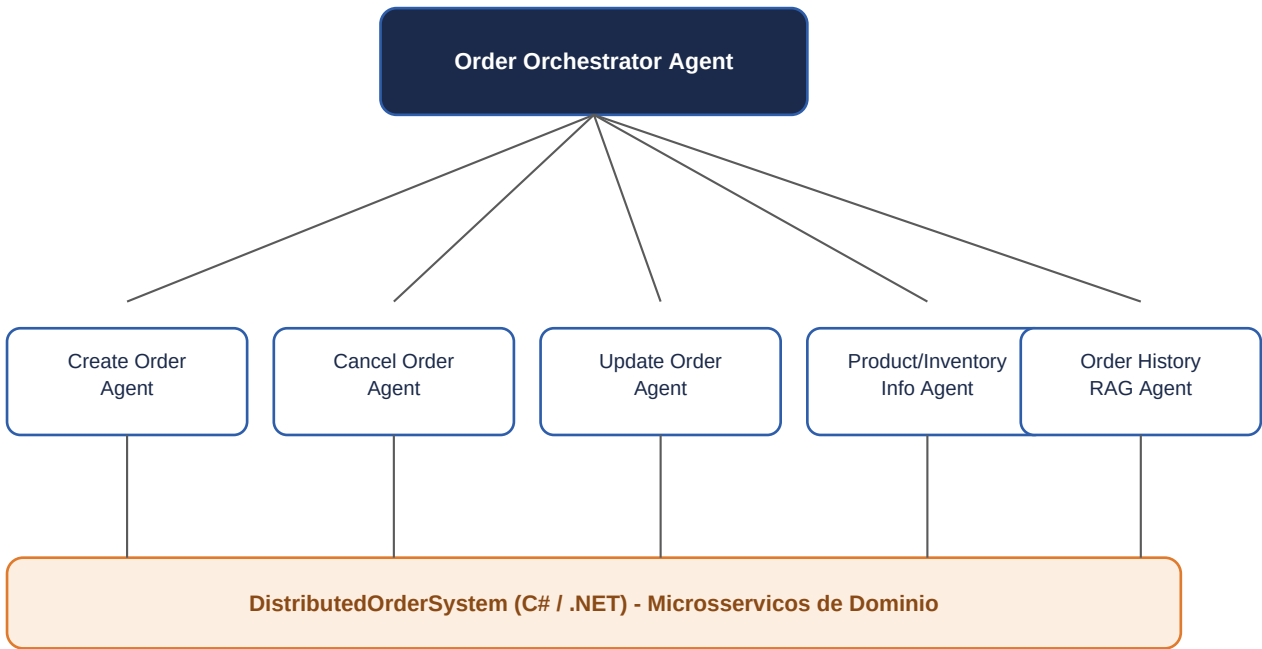
Integração com o Projeto

O DistributedOrderSystem não é um projeto de exemplo descartável - é o backbone técnico de uma futura empresa. A decisão arquitetural central: **Python assume a camada de IA** (agentes, RAG, orquestração) e **C#/.NET permanece intocado como camada de domínio** (regras de negócio, persistência, APIs).

Mapeamento de agentes para o domínio de pedidos

Agente genérico	Agente no projeto
Orchestrator	Order Orchestrator Agent
Booking	Create Order Agent
Cancellation	Cancel Order Agent
Reschedule	Update Order Agent
Information	Product/Inventory Info Agent
RAG/FAQ	Order History RAG Agent

Diagrama multi-agente aplicado ao domínio



Fluxo: usuario -> Orchestrator -> agente especialista -> API .NET do dominio

Figura 3.1 - O Orchestrator recebe a intenção do usuário, roteia para o agente especialista correspondente, que por sua vez chama a API .NET do domínio.

Glossário

LLM (Large Language Model)

Modelo estatístico treinado para prever o próximo token de um texto, usado para gerar respostas, código e raciocínio.

Token

Unidade mínima de texto processada pelo modelo; aproximadamente 3-4 caracteres em português.

Embedding

Representação numérica (vetor) do significado de um texto, usada para medir similaridade semântica.

RAG (Retrieval-Augmented Generation)

Técnica que busca informação relevante em uma base de conhecimento antes de gerar a resposta do modelo.

Agente

Sistema que usa um LLM para decidir ações, chamar ferramentas e iterar até cumprir um objetivo.

Tool Use / Function Calling

Capacidade do modelo de chamar funções externas (ex: consultar um pedido no .NET) durante a conversa.

MCP (Model Context Protocol)

Protocolo aberto para conectar modelos a ferramentas e fontes de dados de forma padronizada.

Orchestrator

Agente responsável por rotear a intenção do usuário para o agente especialista correto.

Fine-tuning

Processo de re-treinar um modelo pré-existente com dados específicos de um domínio.

LoRA / QLoRA

Técnicas eficientes de fine-tuning que ajustam poucos parâmetros extras, reduzindo custo computacional.

Prompt Engineering

Prática de estruturar instruções para obter o melhor comportamento possível do modelo.

Context Window

Quantidade máxima de tokens (entrada + saída) que o modelo consegue processar em uma chamada.

Temperature

Parâmetro que controla a aleatoriedade/criatividade da geração de texto.

System Prompt

Instrução inicial que define papel, tom e regras do assistente antes da conversa do usuário.

Referência de Arquitetura

Visão consolidada do plano de 26 semanas, dividido em 6 fases. Cada fase entrega capacidades concretas dentro do DistributedOrderSystem, sem abstrair prematuramente para um template genérico.

Fase	Semanas	Foco
1 - Fundamentos	1-4	Python, API Anthropic, embeddings, RAG do zero
2 - Agentes e MCP	5-10	Ollama, tool use, MCP server, Semantic Kernel, LangGraph
3 - RAG Avançado	11-16	Hybrid search, reranking, LlamaIndex, RAGAS, DSPy
4 - Fine-tuning	17-22	LoRA/QLoRA com Unsloth, DPO, vLLM, MLOps
5 - Produção + Negócio	23-26	LiteLLM, extração do backbone, instanciação do nicho

Stack consolidada

Camada	Tecnologia	Papel
IA / Orquestração	Python (LangGraph, LlamaIndex, FastMCP, Unsloth, RAGAS, DSPy)	Camada principal de IA
Domínio / APIs	C# / .NET (microsserviços existentes)	Camada de domínio - não mexer
Bridge opcional	Semantic Kernel (.NET)	Ponte .NET <-> Python apenas
LLM local	Ollama (Llama 3.2 / Phi-3.5)	Dev offline, zero custo
LLM cloud	Anthropic API (claude-sonnet-4-6)	Produção
Gateway	LiteLLM	Roteamento cloud/local
Observabilidade	LangSmith / Langfuse	Traces e métricas

Regra anti-abstração prematura: construir o concreto primeiro (Orders, ponta a ponta). Generalizar para o template core/ + packs/ só ao instanciar o segundo domínio, na Fase 5.

Streaming, Histórico e o Primeiro Chatbot

Encerrando a Semana 1, três peças que transformam uma chamada isolada em algo parecido com um chatbot de verdade: entrega incremental da resposta (streaming), memória de curto prazo via reenvio de histórico, e as duas juntas num loop interativo.

Streaming: mesma computação, entrega diferente

Um LLM gera texto de forma **autoregressiva** - token por token - independente de streaming estar ligado ou não. O streaming muda apenas **quando** cada token chega até o cliente: em vez de esperar a resposta inteira pronta, o servidor abre uma única conexão HTTP e vai entregando pedaços conforme são gerados, via **Server-Sent Events (SSE)**. O tempo total até a última palavra é praticamente o mesmo - o que melhora é a **latência percebida**: a primeira palavra aparece quase na hora, em vez de uma tela em branco.

```
# week1/streaming_call.py
with client.messages.stream(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system=SYSTEM_PROMPT,
    messages=[{"role": "user", "content": pergunta}]
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
    final_message = stream.get_final_message()

print(f"Motivo de parada: {final_message.stop_reason}")
```

Trecho essencial - week1/streaming_call.py

Streaming compensa em respostas longas e quando há um humano esperando na tela - é praticamente obrigatório num chat. Em chamadas automatizadas, sem ninguém olhando (agente conversando com outro agente, por exemplo), geralmente não traz benefício e só adiciona complexidade de código.

Histórico de conversa: a API não lembra sozinha

A API é **stateless**: nenhuma chamada sabe da chamada anterior. "Memória" de conversa é responsabilidade do cliente - mantemos uma lista **messages** alternando papéis **user** e **assistant**, e reenviamos essa lista inteira a cada nova pergunta. Isso tem custo visível: numa conversa de teste, a segunda pergunta subiu de 85 para 364 tokens de input - o salto é a pergunta e a resposta anteriores viajando de novo, mais o system prompt.

```
# week1/conversation_history.py
messages = []

def ask(question):
    messages.append({"role": "user", "content": question})
    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=1024,
        system=SYSTEM_PROMPT, messages=messages
    )
    answer = response.content[0].text
```

```
messages.append({"role": "assistant", "content": answer})
return answer
```

Trecho essencial - week1/conversation_history.py

Prompt caching: reduz custo, não reduz envio

Um erro comum: achar que prompt caching evita reenviar a conversa. Não evita - os mesmos bytes trafegam pela rede em toda chamada. O que fica em cache, do lado do servidor, é o **estado computado** (atenção) referente ao trecho fixo do prompt; se o início do prompt for byte-a-byte idêntico ao de uma chamada recente, o modelo reaproveita esse cálculo em vez de refazê-lo. O ganho é em custo (leitura de cache custa uma fração do preço normal) e latência de processamento - não em volume de dados transmitidos.

Chatbot básico: streaming + histórico juntos

O exercício final da semana junta as duas técnicas num loop interativo real - o primeiro protótipo, ainda simples, do que mais tarde vira o Order History RAG Agent no mapeamento de agentes do projeto.

```
# week1/chatbot.py
messages = []
while True:
    user_input = input("Voce: ").strip()
    if user_input.lower() in ("sair", "exit", "quit"):
        break
    messages.append({"role": "user", "content": user_input})
    with client.messages.stream(
        model="claude-sonnet-4-6", max_tokens=1024,
        system=SYSTEM_PROMPT, messages=messages
    ) as stream:
        for text in stream.text_stream:
            print(text, end="", flush=True)
        final_message = stream.get_final_message()
    messages.append({"role": "assistant",
                     "content": final_message.content[0].text})
```

Trecho essencial - week1/chatbot.py

Limitação observada: comportamento não é conhecimento

Rodando o chatbot, o modelo descreveu uma arquitetura de microsserviços plausível, mas avisou por conta própria: "não tenho acesso à documentação/código real do seu repositório". Isso expõe uma distinção importante: **system prompt define comportamento (papel, tom, idioma), não injeta conhecimento real** sobre o DistributedOrderSystem. Duas soluções, para dois problemas diferentes:

- **Conhecimento estático** (arquitetura, políticas, FAQ) → RAG, Semana 4 - buscar trechos reais de documentação/código e injetar no contexto.
- **Dados dinâmicos e transacionais** (status de um pedido específico, estoque em tempo real) → Tool use / MCP, Semana 6-7 - o modelo chama uma ferramenta que consulta a API .NET real, em vez de tentar saber de cor.

Semana 1 concluída: chamada básica, streaming, histórico de conversa e um chatbot funcional - a base mecânica sobre a qual RAG e agentes serão construídos a partir da Semana 4.